(c) For input sequences of length M and output sequences of length N, **what are the complexities of (1) Non-Causal Self-Attention[1] (on the input sequence), (2) Non-Causal Self-Attention (on the input sequence), but with multi-query attention, (3) Cross Attention (from the output sequence to the input sequence), (4) Causal Self-Attention (on the output sequence).** Let d be the hidden dimension of the network (both the encoder and the decoder part), and h be the number of heads.

Recall: Normal multi-headed attention is:

$$\text{MultiHeadedAttention}(Q,K,V) = \text{Concat}_{i\in[\#\text{ heads}]}\left(\text{Attention}\left(XW_i^Q, XW_i^K, XW_i^V\right)\right)W^O$$

For this question, we assume W_i^Q, W_i^K, W_i^V are of shape $d \times d$ and X is shape $M \times d$ where each row of X is a token embedding. We also assume that we use the naive matrix multiplication, that is, if A, B are of shape $m \times n, n \times k$, then AB takes mnk multiply-accumulate operations.

With multi-query attention, there is just one W^K, W^V, thus:

$$\text{MultiQueryAttention}(Q,K,V) = \text{Concat}_{i\in[\#\text{ heads}]}\left(\text{Attention}\left(XW_i^Q, XW^K, XW^V\right)\right)W^O$$

Solution:

i. $\mathcal{O}((Md + M^2)dh)$
ii. $\mathcal{O}((Md + M^2)dh)$
iii. $\mathcal{O}((Md + Nd + MN)hd)$
iv. $\mathcal{O}((Nd + N^2)dh)$

As one example, the encoder-self-attention takes $3Md^2$ operations for each XW_i^Q, XW_i^K, XW_i^V. Computing the attention matrix takes M^2d operations, and $O(M^2)$ for computing its softmax. Multiplying the attention with value matrix takes M^2d operations. This is repeated for h heads. Finally the outputs of all the heads are linearly projected by another hd^2 operations. Together:

$$\mathcal{O}((3Md^2 + M^2d + M^2 + M^2d)h + hd^2) = \mathcal{O}(Md^2h + M^2dh)$$

(i) One potential attention variant is the "polynomial kernel attention", where the similarity function as sim (q, k) is measured by polynomial kernel $\mathcal{K}$ [4]. **Considering a special case for a "quadratic kernel attention" that the degree of "polynomial kernel attention" is set to be 2, derive the sim (q, k) for "quadratic kernel attention". (NOTE: any constant factor is set to be 1.)** .

(ii) One benefit of using kernelized attention is that we can represent a kernel using a feature map $\phi(\cdot)$ [5]. **Derive the corresponding feature map $\phi(\cdot)$ for the quadratic kernel.**

(iii) **Considering a general kernel attention, where the kernel can be represented using feature map that $\mathcal{K}(q, k) = (\phi(q)^T \phi(k))$, rewrite kernel attention of equation 3 with feature map $\phi(\cdot)$.**

(c) We can rewrite the softmax attention in terms of equation 3 as equation 4. **For all the V_i' ($i \in \{1, ..., N\}$), derive the time complexity (asymptotic computational cost) and space complexity (asymptotic memory requirement) of the above softmax attention in terms of sequence length N, D and M.**

NOTE: for memory requirement, we need to store any intermediate results for backpropagation, including all Q, K, V

(d) Assume we have a kernel $\mathcal{K}$ as the similarity function and the kernel can be represented with a feature map $\phi(\cdot)$, we can rewrite equation 3 with sim $(x, y) = \mathcal{K}(x, y) = (\phi(Q_i)^T \phi(K_j))$ in part (b). We can then further simplify it by making use of the associative property of matrix multiplication to

$$V_i' = \frac{\phi(Q_i)^T \sum_{j=1}^{N} \phi(K_j) V_j^T}{\phi(Q_i)^T \sum_{j=1}^{N} \phi(K_j)}. \tag{5}$$

Note that the feature map $\phi(\cdot)$ is applied row-wise to the matrices Q and K.

Considering using a linearized polynomial kernel $\phi(x)$ of degree 2, and assume $M \approx D$, derive the computational cost and memory requirement of this kernel attention as in (5).

1. Kernelized Linear Attention (Part II)

This is the second part of of the "Kernelized Linear Attention" problem you saw previously. Please refer to this part for notation and context.

In Part I of this problem, we considered ways to efficiently express the attention operation when sequences are long (e.g. a long document). Attention uses the following equation:

$$V'_{i\cdot} = \frac{\sum_{j=1}^{N} \text{sim}\left(Q_{i\cdot}, K_{j\cdot}\right) V_{j\cdot}}{\sum_{j=1}^{N} \text{sim}\left(Q_{i\cdot}, K_{j\cdot}\right)}. \tag{1}$$

We saw that when the similarity function is a kernel function (i.e. if we can write $\text{sim}\left(Q_{i\cdot}, K_{j\cdot}\right) = \Phi(Q_{i\cdot})\Phi(K_{j\cdot})^T$ for some function Φ), then we can use the associative property of matrix multiplication to simplify the formula to

$$V'_{i\cdot} = \frac{\phi\left(Q_{i\cdot}\right) \sum_{j=1}^{N} \phi\left(K_{j\cdot}\right)^T V_{j\cdot}}{\phi\left(Q_{i\cdot}\right) \sum_{j=1}^{N} \phi\left(K_{j\cdot}\right)^T}. \tag{2}$$

If we use a polynomial kernel with degree 2, this gives a computational cost of $\mathcal{O}\left(ND^2M\right)$, which for very large N is favorable to the softmax attention computational cost of $\mathcal{O}\left(N^2 \max\left(D, M\right)\right)$. ($N$ is the sequence length, D is the feature dimension of the queries and keys, and M is the feature dimension of the values. Now, we will see whether we can use kernel attention to directly approximate the softmax attention:

$$V'_{i\cdot} = \frac{\sum_{j=1}^{N} \exp(\frac{Q_{i\cdot}K_{j\cdot}^T}{\sqrt{D}}) V_{j\cdot}}{\sum_{j=1}^{N} \exp(\frac{Q_{i\cdot}K_{j\cdot}^T}{\sqrt{D}})}. \tag{3}$$

(a) Approximating softmax attention with linearized kernel attention

i. As a first step, we can use Gaussian Kernel $\mathcal{K}_{\text{Gauss}}(q, k) = \exp(\frac{-||q-k||_2^2}{2\sigma^2})$ to rewrite the softmax similarity function, where $\text{sim}_{\text{softmax}}(q, k) = \exp(\frac{q^T k}{\sqrt{D}})$. Assuming we can have $\sigma^2 = \sqrt{D}$, **rewrite the softmax similarity function using Gaussian Kernel.**. (*Hint: You can write the softmax* $\exp(\frac{-||q-k||_2^2}{2\sigma^2})$ *as the product of the Gaussian Kernel and two other terms.)*

ii. However, writing softmax attention using a Gaussian kernel does not directly enjoy the benefits of the reduced complexity using the feature map. This is because the feature map of Guassian kernel usually comes from the Taylor expression of $\exp(\cdot)$, whose computation is still expensive

[1]. However, we can approximate the Guassian kernel using random feature map and then reduce the computation cost, using a trick from Random feature attention (2021)[2]. (Rahimi and Recht, 2007)[3] proposed random Fourier features to approximate a desired shift-invariant kernel. The method nonlinearly transforms a pair of vectors q and k using a **random feature map** $\phi_{\text{random}}()$; the inner product between $\phi(q)$ and $\phi(k)$ approximates the kernel evaluation on q and k. More precisely:

$$\phi_{\text{random}}(q) = \sqrt{\frac{1}{D_{\text{random}}}}\left[\sin\left(\mathbf{w}_1 q\right), \ldots, \sin\left(\mathbf{w}_{D_{\text{random}}} q\right), \cos\left(\mathbf{w}_1 q\right), \ldots, \cos\left(\mathbf{w}_{D_{\text{random}}} q\right)\right]^{\top}. \tag{4}$$

Where we have D_{random} of D-dimensional random vectors $\mathbf{w}_{i\cdot}$ independently sampled from $\mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_D)$,

$$\mathbb{E}_{\mathbf{w}_{i\cdot}}\left[\phi(q) \cdot \phi(k)\right] = \exp\left(-\|q-k\|^2 / 2\sigma^2\right). \tag{5}$$

Use ϕ_{random} to approximate the above softmax similarity function with Gaussian Kernel and derive the computation cost for computing all the V' here.

(b) (Optional) Beyond self-attention, an autoregressive case will be masking the attention computation such that the i-th position can only be influenced by a position j if and only if $j \leq i$, namely a position cannot be influenced by the subsequent positions. This type of attention masking is called *causal masking*.

Derive how this causal masking changes equation 1 and 2. **Write equation 1 and 2 in terms of $S_{i\cdot}$ and $Z_{i\cdot}$**, which are defined as:

$$S_{i\cdot} = \sum_{j=1}^{i} \phi\left(K_{j\cdot}\right)^T V_{j\cdot}, \; Z_{i\cdot} = \sum_{j=1}^{i} \phi\left(K_{j\cdot}\right)^T, \tag{6}$$

to simplify the causal masking kernel attention and derive the computational complexity of this new causal masking formulation scheme.